CASE STUDY 23: Student Timetable Conflict Detection

Python Activity-Based Learning | Case 1 of 10

Problem statement

Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

Objectives

Validate course records; compare every pair of classes on the same day; report overlapping periods and room unavailability; state whether the timetable is valid.

Assumptions / rules

Each course is stored as (course, day, start period, end period, room). • Periods are inclusive; two classes overlap when their period ranges intersect on the same day. • Room R104 is treated as unavailable in the sample.

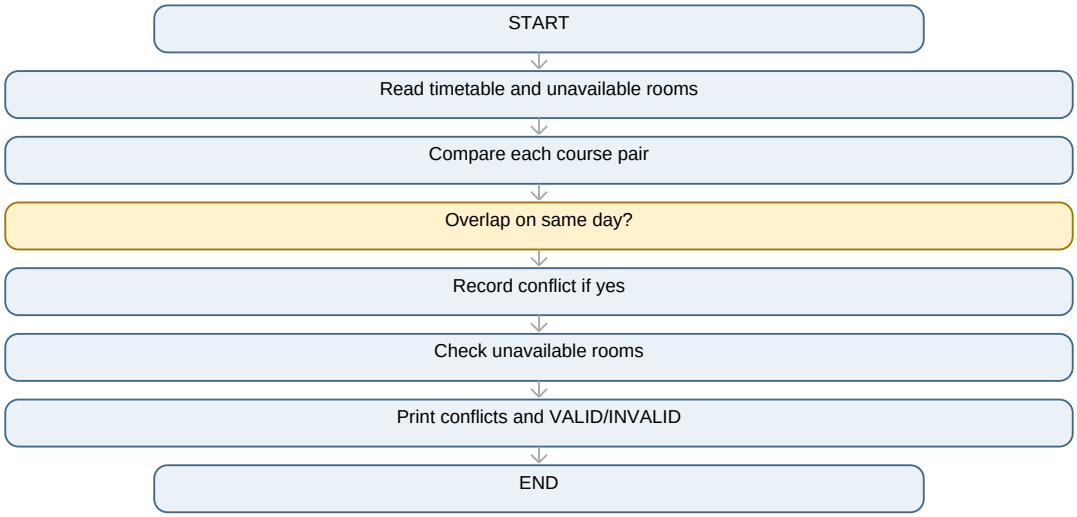
Algorithm

1. Start with the timetable and unavailable-room list. 2. Compare each pair of courses. 3. If the day is the same and periods overlap, record a conflict. 4. Check whether any course uses an unavailable room. 5. Print conflicts and final validity.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 23: Sample Output and Conclusion

Student Timetable Conflict Detection

Expected result / output summary: The sample identifies the Monday overlap between Python (periods 1–2) and Maths (periods 2–3).

Program output (from combined Python source)

```
======================================================================
23. STUDENT TIMETABLE CONFLICT DETECTION
======================================================================
Course timetable:
  Python   Mon periods 1-2, room R101
  Maths    Mon periods 2-3, room R102
  English  Mon periods 4-5, room R101
  Physics  Tue periods 1-2, room R103
CONFLICT: Python overlaps with Maths on Mon.
Unavailable-room assignments: None
Timetable status: INVALID
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 28: Repeated Transaction Detection

Python Activity-Based Learning | Case 2 of 10

Problem statement

Process transaction identifiers and amounts in a list, detect duplicate transactions, identify threshold violations, and generate a list of suspicious entries.

Objectives

Flag duplicate IDs, negative amounts, and amounts above a selected threshold.

Assumptions / rules

Transaction data is stored as (transaction ID, amount). • The sample threshold is Rs. 5,000. • A transaction may have more than one suspicious reason.

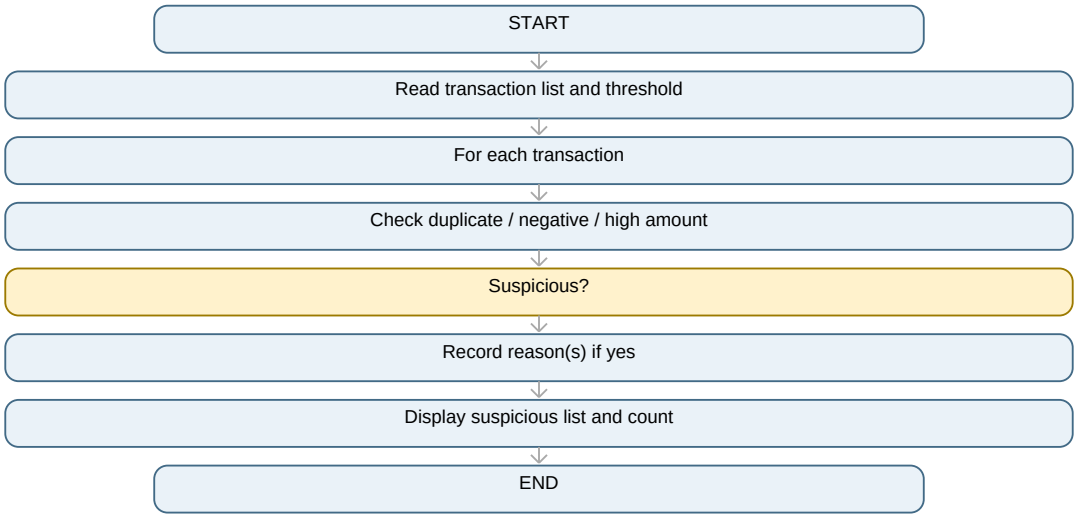
Algorithm

1. Create an empty list of seen IDs and suspicious records. 2. Visit each transaction. 3. Check for a duplicate ID, negative amount, or amount above threshold. 4. Store and display each suspicious record and its reasons.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 28: Sample Output and Conclusion

Repeated Transaction Detection

Expected result / output summary: The repeated TX101, TX103 above the threshold, and TX104 negative amount are flagged.

Program output (from combined Python source)

```
======================================================================
28. REPEATED TRANSACTION DETECTION
======================================================================
Transaction records:
  TX101: Rs. 450
  TX102: Rs. 1200
  TX101: Rs. 450
  TX103: Rs. 9500
  TX104: Rs. -50
  TX105: Rs. 700
Suspicious entries:
  TX101 | Rs. 450 | duplicate ID
  TX103 | Rs. 9500 | above threshold
  TX104 | Rs. -50 | negative amount
Total records: 6
Suspicious records: 3
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 30: Student Email and Identifier Validation

Python Activity-Based Learning | Case 3 of 10

Problem statement

Validate student email-like strings and registration identifiers against formatting rules, reporting malformed separators, invalid character groups, and missing components.

Objectives

Validate email structure and registration ID length and character groups; print a reason for invalid records.

Assumptions / rules

Email validation is a basic format check. • Registration IDs must contain 3 uppercase letters followed by 7 digits. • The checks do not contact a mail server or verify a real student record.

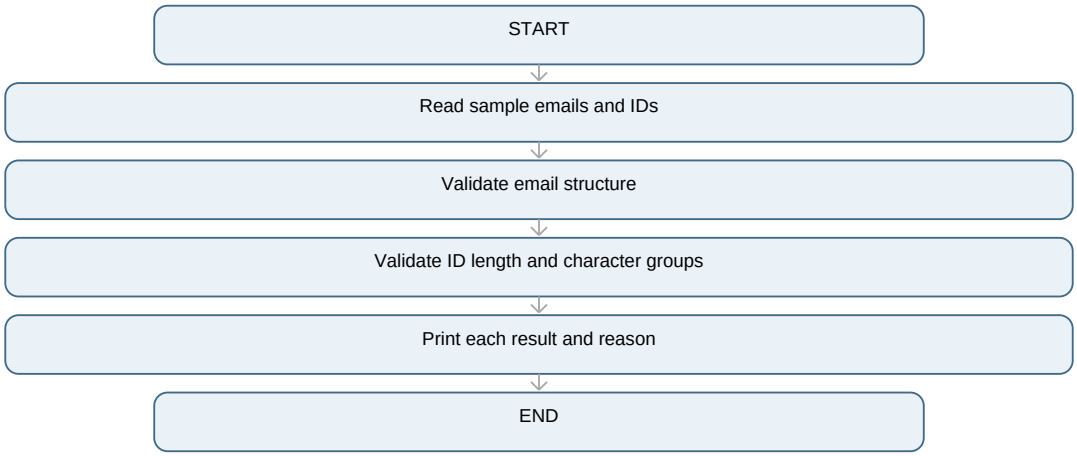
Algorithm

1. For each email, require exactly one @, non-empty local and domain parts, a dot in the domain, and permitted local-part characters. 2. For each registration ID, check length 10, first three uppercase letters, and remaining seven digits. 3. Print VALID or INVALID with a reason.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 30: Sample Output and Conclusion

Student Email and Identifier Validation

Expected result / output summary: The sample output distinguishes valid formats from missing @, malformed separators, and invalid registration IDs.

Program output (from combined Python source)

```
======================================================================
30. STUDENT EMAIL AND IDENTIFIER VALIDATION
======================================================================
Email checks:
  rehan23@college.edu        -> VALID (valid format)
  aisha.k@college.edu        -> VALID (valid format)
  bad-email                  -> INVALID (must contain exactly one @)
  student@@college.edu       -> INVALID (must contain exactly one @)
  name@college               -> INVALID (domain must contain a dot in a valid position)
Registration ID checks:
  CSE2026001     -> VALID (valid format)
  ECE2026042     -> VALID (valid format)
  CS2026         -> INVALID (must be 10 characters)
  CSE20A6003     -> INVALID (last 7 characters must be digits)
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 38: ATM Menu-Driven Banking Simulation

Python Activity-Based Learning | Case 4 of 10

Problem statement

Implement repeated menu operations for deposits, withdrawals, balance inquiries, and transaction summaries, with validation and specified account limits.

Objectives

Simulate deposit, withdrawal, balance, and summary operations; reject invalid amounts, insufficient balance, and withdrawals beyond the daily limit.

Assumptions / rules

Opening balance is Rs. 5,000. • Daily withdrawal limit is Rs. 3,000. • The sample operation sequence is predefined for repeatable output.

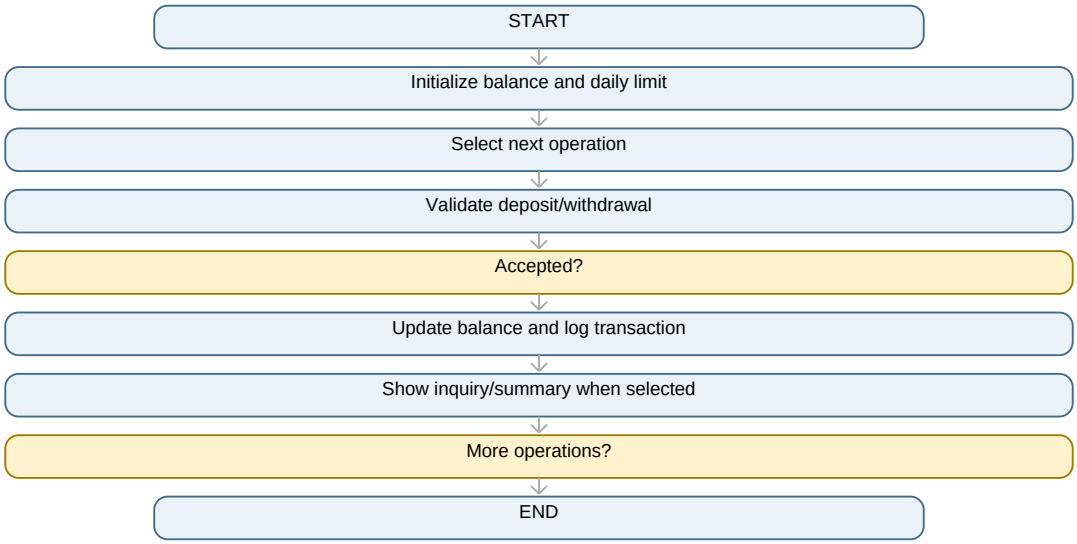
Algorithm

1. Initialize balance, daily withdrawal total, and transaction list. 2. Process each sample operation. 3. Validate amount and available balance/limit. 4. Update balance for accepted transactions. 5. Display balance and transaction summary.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 38: Sample Output and Conclusion

ATM Menu-Driven Banking Simulation

Expected result / output summary: The second withdrawal is rejected because it would exceed the daily withdrawal limit.

Program output (from combined Python source)

```
======================================================================
38. ATM MENU-DRIVEN BANKING SIMULATION
======================================================================
Deposit Rs. 1500.00: accepted
Withdrawal Rs. 1200.00: accepted
Withdrawal Rs. 2500.00: rejected (daily limit exceeded)
Balance inquiry: Rs. 5300.00
Transaction summary:
  Deposit    Rs.  1500.00 - Accepted
  Withdrawal Rs.  1200.00 - Accepted
  Withdrawal Rs.  2500.00 - Rejected
Final balance: Rs. 5300.00
Total withdrawn today: Rs. 1200.00
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 66: Student Academic Risk Classification

Python Activity-Based Learning | Case 5 of 10

Problem statement

Combine lists of marks, attendance, and laboratory scores to classify students into risk categories, identify multiple simultaneous risk factors, and generate a summary.

Objectives

Calculate each student's average marks, count risk factors, classify risk, and summarize category totals.

Assumptions / rules

Risk factors: average marks below 40, attendance below 75%, laboratory score below 50%. • Two or more factors means HIGH risk; one means MEDIUM; none means LOW.

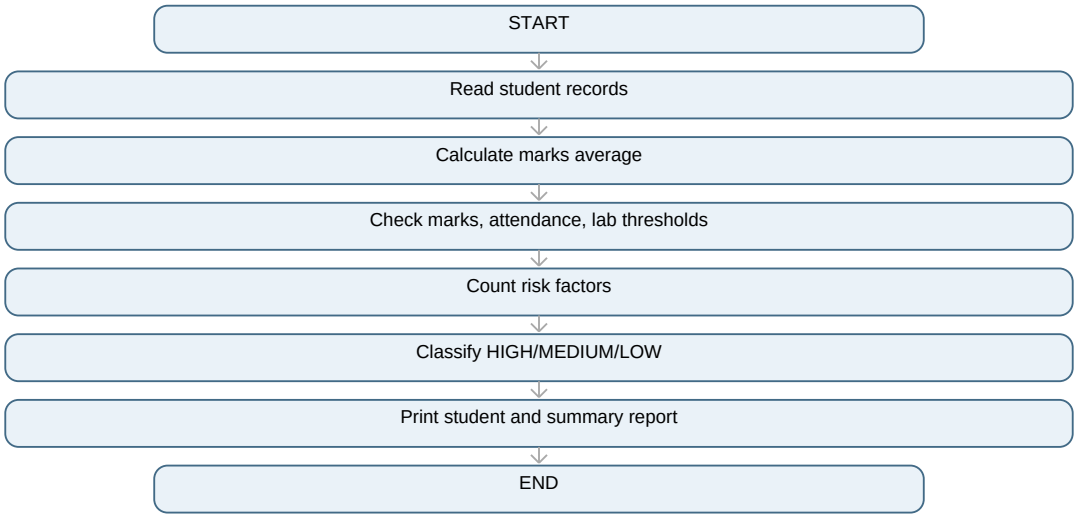
Algorithm

1. Read each student's marks, attendance, and lab score. 2. Calculate average marks. 3. Check all three risk conditions. 4. Classify according to the number of risk factors. 5. Print student details and overall counts.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 66: Sample Output and Conclusion

Student Academic Risk Classification

Expected result / output summary: The output lists each student's risk factors and the number of students in each category.

Program output (from combined Python source)

```
======================================================================
66. STUDENT ACADEMIC RISK CLASSIFICATION
======================================================================
Risk rules: marks average < 40, attendance < 75%, lab score < 50%.
Asha     average= 77.0, attendance= 92%, lab= 85%
         risk=LOW; factors=none
Bharat   average= 43.0, attendance= 68%, lab= 45%
         risk=HIGH; factors=low attendance, low laboratory score
Charan   average= 58.0, attendance= 76%, lab= 52%
         risk=LOW; factors=none
Divya    average= 39.7, attendance= 82%, lab= 38%
         risk=HIGH; factors=low marks, low laboratory score
Summary: HIGH=2, MEDIUM=0, LOW=2
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 72: Smart Hostel Resource Monitoring

Python Activity-Based Learning | Case 6 of 10

Problem statement

Analyse room-wise electricity and water consumption, calculate charges, identify threshold violations, and compare resource utilization across floors.

Objectives

Calculate electricity and water costs per room, flag high usage, and produce floor-wise totals.

Assumptions / rules

Electricity rate is Rs. 8 per kWh. • Water rate is Rs. 20 per 1,000 litres. • Thresholds: 180 kWh and 2,000 litres per room.

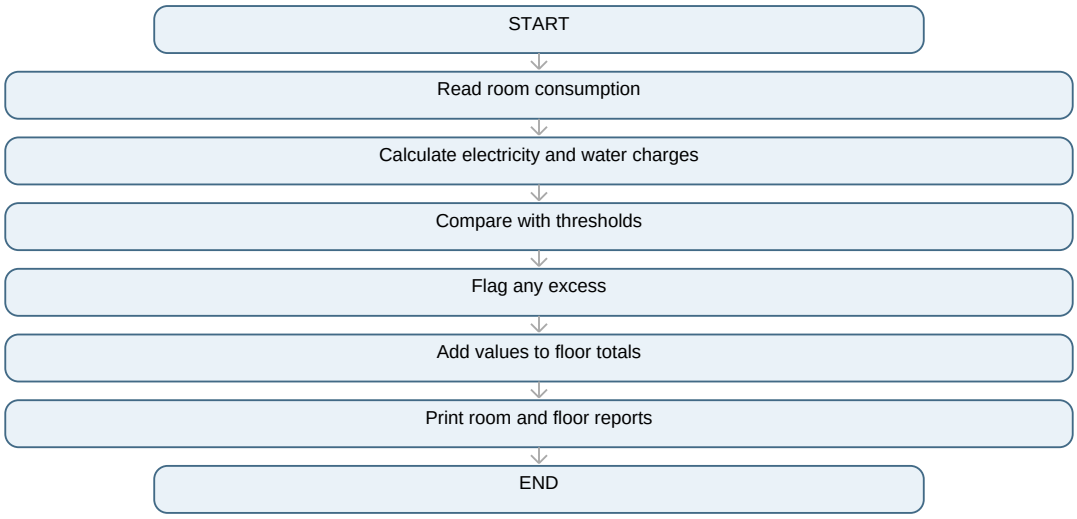
Algorithm

1. Read each room's floor, electricity use, and water use. 2. Calculate electricity and water charges. 3. Compare usage against thresholds and flag violations. 4. Accumulate totals by floor. 5. Print room bills and floor summary.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 72: Sample Output and Conclusion

Smart Hostel Resource Monitoring

Expected result / output summary: The sample flags rooms that exceed the electricity or water threshold and prints floor totals.

Program output (from combined Python source)

```
======================================================================
72. SMART HOSTEL RESOURCE MONITORING
======================================================================
Room 101 (floor 1): 145 kWh, 1800 L, bill=Rs. 1196.00; alerts=none
Room 102 (floor 1): 95 kWh, 1200 L, bill=Rs. 784.00; alerts=none
Room 201 (floor 2): 210 kWh, 2400 L, bill=Rs. 1728.00; alerts=electricity high, water high
Room 202 (floor 2): 130 kWh, 1600 L, bill=Rs. 1072.00; alerts=none
Floor summary:
  Floor 1: 240 kWh, 3000 L, total charges Rs. 1980.00
  Floor 2: 340 kWh, 4000 L, total charges Rs. 2800.00
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 76: University Examination Result Moderation

Python Activity-Based Learning | Case 7 of 10

Problem statement

Process marks across multiple subjects, apply specified moderation rules, preserve maximum-mark constraints, and compare original and moderated outcomes.

Objectives

Apply grace marks under a stated rule, cap marks at 100, and compare pass/fail results before and after moderation.

Assumptions / rules

Pass mark is 40 in every subject. • Three grace marks are added only when a mark is below 40. • Marks cannot exceed 100.

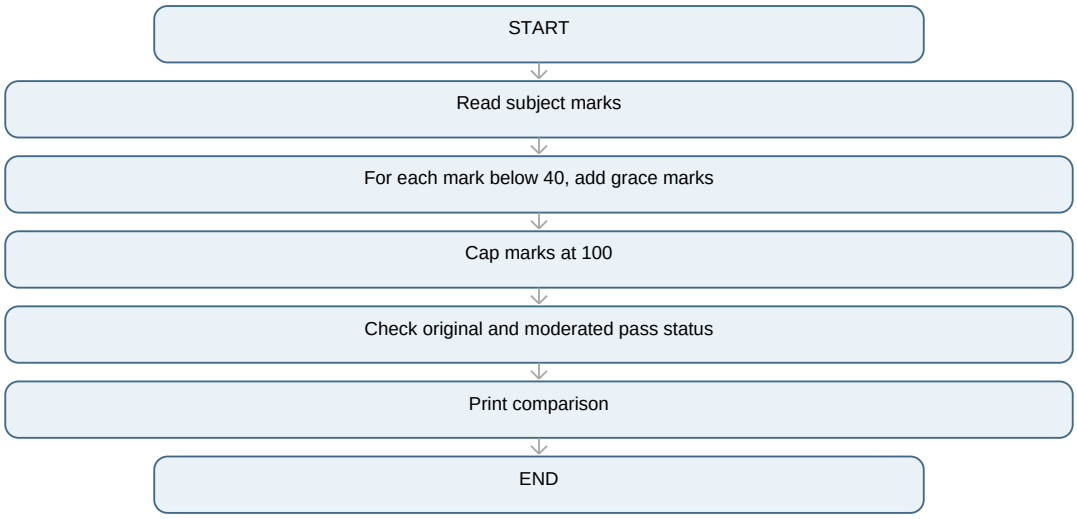
Algorithm

1. For each student, copy the original marks. 2. Add 3 to marks below 40 and cap at 100. 3. Check whether all original marks meet the pass mark. 4. Check whether all moderated marks meet the pass mark. 5. Print both lists and results.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 76: Sample Output and Conclusion

University Examination Result Moderation

Expected result / output summary: The output shows original marks, moderated marks, and pass/fail status before and after moderation.

Program output (from combined Python source)

```
======================================================================
76. UNIVERSITY EXAMINATION RESULT MODERATION
======================================================================
Asha: original=[38, 67, 81], moderated=[41, 67, 81]
  Result before moderation: FAIL
  Result after moderation:  PASS
Bharat: original=[29, 45, 54], moderated=[32, 45, 54]
  Result before moderation: FAIL
  Result after moderation:  FAIL
Charan: original=[49, 52, 61], moderated=[49, 52, 61]
  Result before moderation: PASS
  Result after moderation:  PASS
Rule: 3 grace marks are added only to subjects below 40; maximum is 100.
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 77: Smart Electricity Billing for Multiple Apartments

Python Activity-Based Learning | Case 8 of 10

Problem statement

Calculate apartment-wise bills using progressive tariffs, fixed charges, and rebates; identify billing discrepancies and rank apartments by consumption.

Objectives

Calculate slab-wise charges, add a fixed fee, apply the low-usage rebate, and rank apartments by units consumed.

Assumptions / rules

First 100 units cost Rs. 3 per unit; next 100 cost Rs. 5 per unit; units above 200 cost Rs. 7 per unit. • Fixed charge is Rs. 50. • A Rs. 20 rebate applies below 60 units.

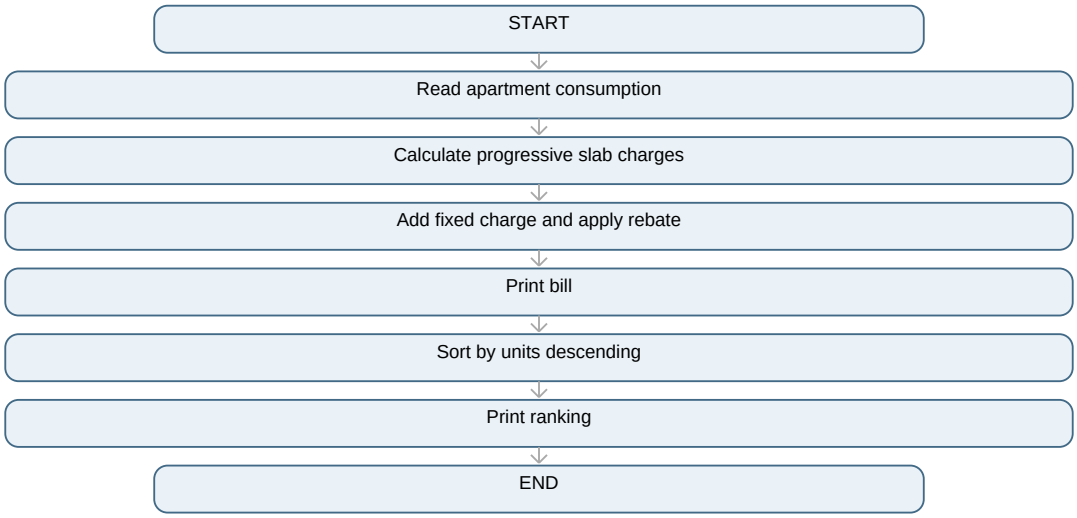
Algorithm

1. Read each apartment's consumption. 2. Calculate charges for each tariff slab. 3. Add the fixed charge and subtract any eligible rebate. 4. Print the bill. 5. Sort apartments by consumption in descending order.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 77: Sample Output and Conclusion

Smart Electricity Billing for Multiple Apartments

Expected result / output summary: The output shows itemized bills and ranks apartments from highest to lowest consumption.

Program output (from combined Python source)

```
======================================================================
77. SMART ELECTRICITY BILLING FOR MULTIPLE APARTMENTS
======================================================================
A-101: 85 units; energy=Rs. 255.00; fixed=Rs. 50; rebate=Rs. 0; bill=Rs. 305.00
A-102: 145 units; energy=Rs. 525.00; fixed=Rs. 50; rebate=Rs. 0; bill=Rs. 575.00
B-201: 230 units; energy=Rs. 1010.00; fixed=Rs. 50; rebate=Rs. 0; bill=Rs. 1060.00
B-202: 55 units; energy=Rs. 165.00; fixed=Rs. 50; rebate=Rs. 20; bill=Rs. 195.00
Apartments ranked by consumption (highest first):
  B-201: 230 units
  A-102: 145 units
  A-101: 85 units
  B-202: 55 units
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 81: Campus Event Seating and Entry Validation

Python Activity-Based Learning | Case 9 of 10

Problem statement

Generate seating assignments from nested lists, validate tickets against registration identifiers, detect duplicate entries, and report available seating capacity.

Objectives

Display a seating matrix, validate ticket entries, reject duplicates or unseated registrations, and calculate vacant seats.

Assumptions / rules

The seating plan is a nested list; EMPTY represents a vacant seat. • A ticket is accepted only if the registration is seated and has not already entered. • Capacity is the total number of positions in the matrix.

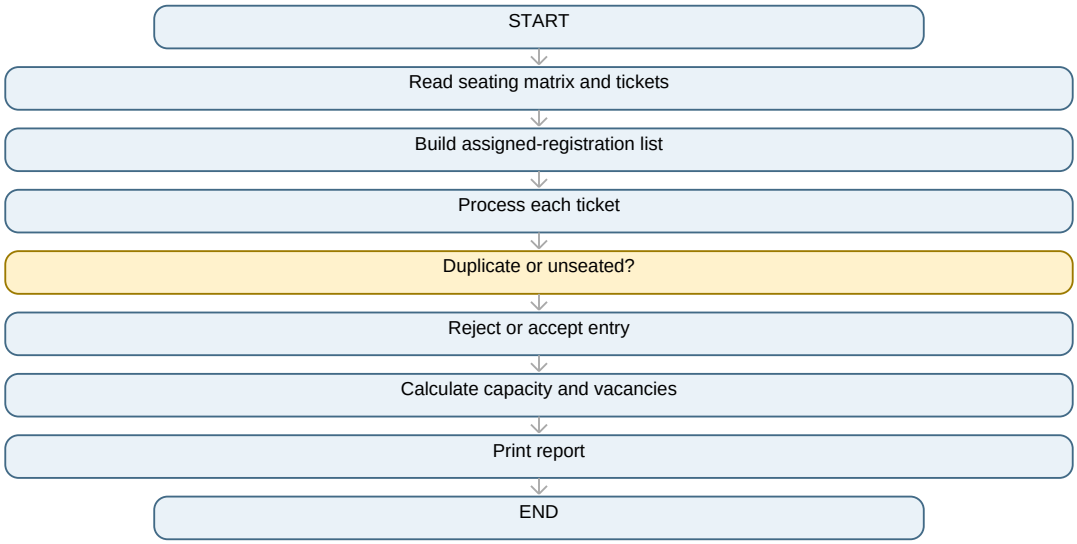
Algorithm

1. Flatten the seating matrix to find assigned registrations. 2. Process each ticket. 3. Reject tickets already seen or not in the seating list. 4. Accept valid first entries. 5. Count capacity, occupied seats, vacant seats, and duplicate assignments.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 81: Sample Output and Conclusion

Campus Event Seating and Entry Validation

Expected result / output summary: The sample rejects a repeated REG102 entry and an unseated REG999 ticket, then reports seating capacity.

Program output (from combined Python source)

```
======================================================================
81. CAMPUS EVENT SEATING AND ENTRY VALIDATION
======================================================================
Seat map:
  Row 1: REG101 | REG102 | EMPTY
  Row 2: REG103 | REG104 | REG105
  Row 3: EMPTY | REG106 | REG102
Ticket validation:
  REG101: ACCEPTED
  REG102: ACCEPTED
  REG103: ACCEPTED
  REG102: REJECTED (duplicate entry)
  REG999: REJECTED (registration not seated)
Total capacity: 9
Occupied seats: 7
Available seats: 2
Duplicate seat assignments: ['REG102']
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.

CASE STUDY 94: Disaster Relief Supply Allocation

Python Activity-Based Learning | Case 10 of 10

Problem statement

Compare stock lists against area-wise requirements, prioritize urgent shortages, allocate limited supplies according to a prescribed policy, and calculate remaining unmet demand.

Objectives

Allocate available stock to the most urgent requests first and report remaining stock and unmet demand.

Assumptions / rules

Priority 1 is most urgent; lower priority numbers are processed first. • Stock cannot become negative. • When stock is insufficient, allocate what remains and record the shortfall.

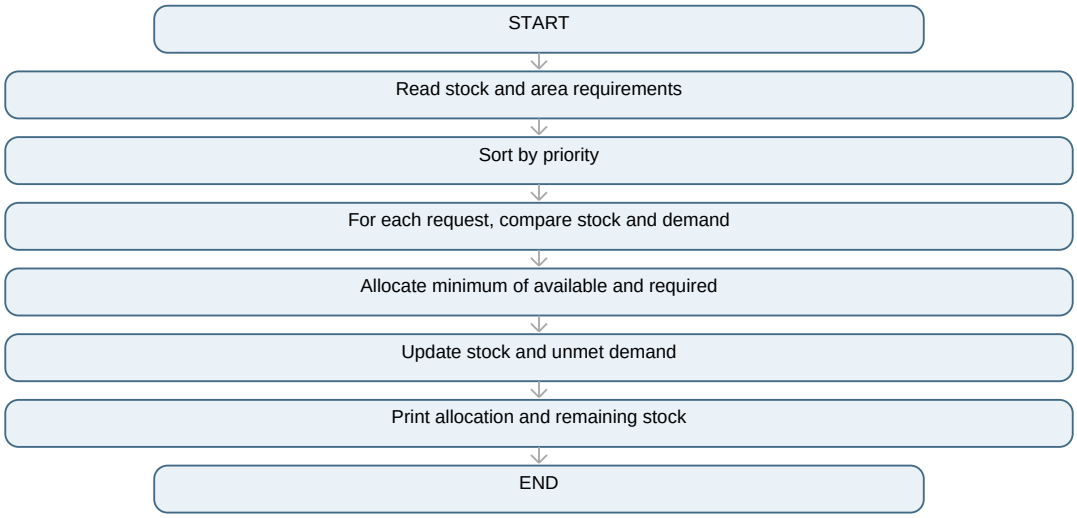
Algorithm

1. Initialize available stock by supply item. 2. Sort requests by priority. 3. For each request, allocate the smaller of stock available and quantity required. 4. Reduce stock and calculate unmet demand. 5. Print allocation, remaining stock, and total unmet demand.

Pseudocode

START → read sample data → process records using the stated rules → display results → STOP

Flowchart



CASE STUDY 94: Sample Output and Conclusion

Disaster Relief Supply Allocation

Expected result / output summary: The output gives allocated quantities, unmet demand by area/item, and remaining stock.

Program output (from combined Python source)

```
======================================================================
94. DISASTER RELIEF SUPPLY ALLOCATION
======================================================================
Allocation policy: priority 1 is most urgent; allocate available stock in priority order.
Area A  | Water      | priority 1 | needed  70 | allocated  70 | unmet   0
Area A  | Food packs | priority 1 | needed  50 | allocated  50 | unmet   0
Area B  | Blankets   | priority 1 | needed  30 | allocated  30 | unmet   0
Area B  | Water      | priority 2 | needed  50 | allocated  30 | unmet  20
Area C  | Food packs | priority 2 | needed  45 | allocated  30 | unmet  15
Area C  | Blankets   | priority 3 | needed  20 | allocated  10 | unmet  10
Remaining stock:
  Water: 0
  Food packs: 0
  Blankets: 0
Total unmet demand: 45

All 10 case-study demonstrations completed.
```

Conclusion

This case study demonstrates how basic Python logic can process the sample data and report results under the stated assumptions. Replace sample values or rules if your faculty specifies different requirements.